

Vaidya

Production Perfect Low-Level Design and Implementation Contract

Text-first clinic receptionist engine, admin/doctor application, and voice-ready architecture

Decision area	Final decision
Backend stack	NestJS + TypeScript using Fastify adapter. This is the final recommended backend because we are starting from scratch and want one language across frontend and backend, strong modular architecture, DI, WebSocket support, queues, typed contracts, and production maintainability.
Frontend stack	Next.js + TypeScript + Tailwind CSS for the admin/doctor web application.
Validation	Zod schemas shared between backend and frontend where possible. Nest pipes/adapters use Zod validation.
Database	PostgreSQL as source of truth. Supabase Pro or managed Postgres for first production pilot.
ORM/query layer	Drizzle ORM + SQL migrations as source of truth. Raw SQL migrations are allowed for exclusion constraints, pgvector, triggers, and advanced indexes.
Auth	Supabase Auth or equivalent JWT provider for login; application RBAC is enforced in backend using clinic_users and doctor ownership.
Queue/cache	BullMQ + Redis/Valkey for notifications, cleanup jobs, future voice events, and background tasks. In-memory fallback only for local development.
Voice provider	Provider adapter pattern. Exotel is likely first India pilot option; Twilio can be supported through the same interface. Do not hardcode one provider.
LLM models	Sarvam 30B provider adapter for primary intent/entity/service routing; Sarvam 105B fallback for low-confidence/ambiguous cases. Model IDs and providers remain configurable.
Voice models	Sarvam STT/TTS provider adapters for Tamil/Tanglish and English. Voice comes after text QA is stable.
Booking mode	Admin configurable. Default is pending confirmation. Auto-confirm can be enabled by admin later.
Retention	MP3/audio retained 10 days. Transcript text retained 30 days. Appointment records retained as operational history.

1. Executive Decision: Backend Stack

Because this project is starting from scratch, the backend decision should optimize for production maintainability, type safety, state-machine correctness, frontend/backend contract sharing, and a small two-person team. The recommended backend is NestJS with TypeScript and the Fastify adapter.

Option	Strengths	Weakness for Vaidya	Decision
FastAPI + Python	Excellent API framework, Pydantic validation, strong Python ecosystem, simple async APIs.	Requires Python backend plus TypeScript frontend. Shared schemas/types become harder. Voice and AI calls are external APIs, so Python ML ecosystem is not a major advantage. The team must maintain two language stacks.	Good option if the team is Python-first, but not the final recommendation.
Bare Fastify + TypeScript	Very fast, lightweight, direct control, Zod works well, easy to build small services.	Can become under-structured as flows grow unless discipline is strong. More manual module boundaries and DI.	Good for MVP, but less ideal than NestJS for a production modular monolith.
NestJS + Fastify adapter + TypeScript	Modular architecture, DI, guards, interceptors, validation pipeline, testing patterns, WebSocket support, queues, and strong TypeScript integration with Next.js.	Slightly more boilerplate than bare Fastify.	Chosen backend. Best suited for Vaidya production from scratch.

Final backend decision: NestJS + TypeScript + Fastify adapter. Use a modular monolith first, not microservices. Use modules for auth, conversation, appointments, clinic setup, knowledge, calls, notifications, and audit.

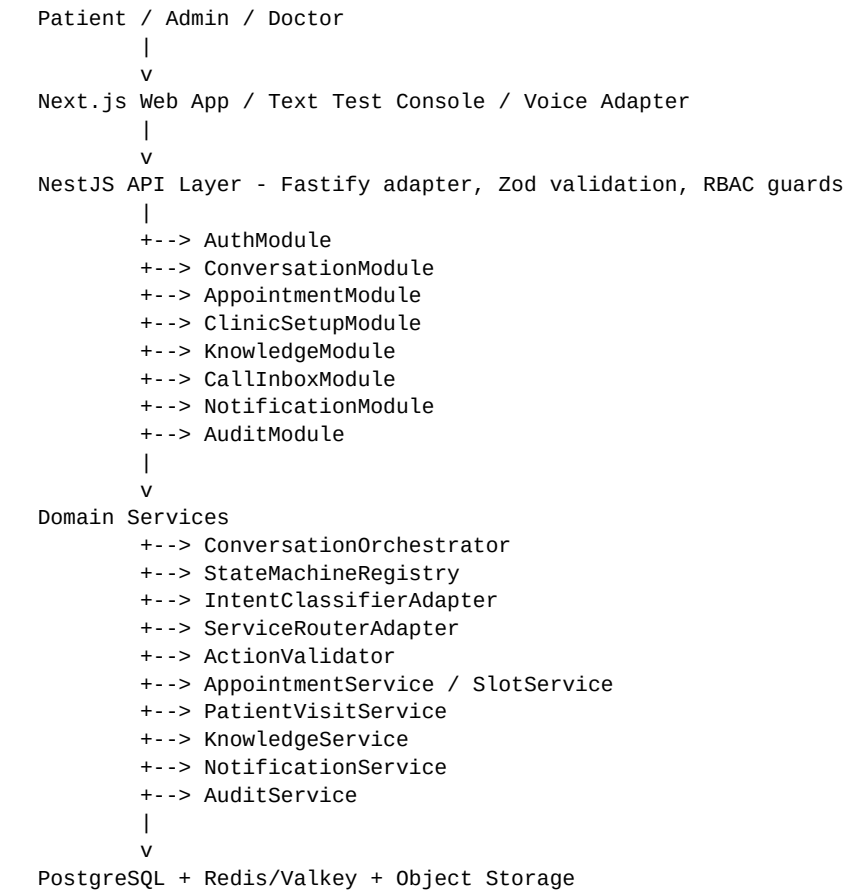
Important correction to the earlier implementation plan: do not mix FastAPI with Zod. If using FastAPI, use Pydantic. Since the final stack is TypeScript/NestJS, use Zod or TypeBox and share schemas with the frontend where practical.

2. Product Scope and Non-Negotiable Principles

- **Text first:** All core flows are implemented and tested as text-to-text before voice integration.
- **Voice later:** Voice is a channel wrapper around the same conversation engine. STT/TTS do not own product logic.
- **Agent safety:** The LLM never writes to the database, never deletes records, and never executes actions directly.
- **Backend ownership:** Backend state machines, domain services, validators, and database transactions own all business actions.
- **Clinic control:** Pending confirmation is default. Admin can later enable auto-confirm for trusted clinics.
- **Structured source of truth:** Fees, timings, doctor schedules, holidays, locations, services, and slots come from structured DB tables, not knowledge-base text.
- **Approved knowledge only:** Knowledge base is for clinic-approved Q&A. If an approved answer is missing, Vaidya says clinic staff will confirm.
- **No hard delete:** Operational records are cancelled, disabled, archived, or expired. Normal UI does not hard-delete.
- **Auditability:** Every critical admin, doctor, agent, appointment, notification, and safety action is audited.

3. Corrected Production Architecture

The architecture is a modular monolith with clean internal boundaries. Application servers are stateless. PostgreSQL stores durable state. Redis/Valkey handles queues, locks, and background jobs after production traffic starts. Object storage stores audio recordings, never Postgres.



Layer	Responsibility	Production notes
Frontend	Admin and doctor portal, local text test console, role-specific views.	No direct DB access. All operations use backend APIs.
API layer	HTTP endpoints, auth, RBAC, validation, request tracing, error mapping.	Reject invalid clinic_id, cross-tenant access, unsupported actions, and missing idempotency keys.
Agent layer	Intent/entity extraction, service routing, language detection, safe response planning.	Agent returns structured JSON only. No SQL, no write actions.
State machines	Booking, cancel, reschedule, handoff, fee clarification, FAQ side questions, emergency handling.	State stored in conversation_sessions.current_flow/current_state/collected_json.
Domain services	Appointments, slots, patient visits, knowledge, notifications, calls, audit.	All writes happen through transactions and validators.
PostgreSQL	Source of truth for clinic data and operational records.	Use composite FKs, row locks, unique indexes, exclusion constraints, status transitions.
Object storage	MP3 recordings.	10-day retention; DB stores metadata and signed URL pointer only.
Queue	Notifications, cleanup, async jobs, future voice events.	Retry with backoff. Dead-letter queue later.

4. Runtime Modes

Mode	Input	Output	Use
Text test console	Typed patient message.	Typed Vaidya reply.	Primary QA and regression testing.
Admin/doctor web app	Authenticated user actions.	Operational data and actions.	Production app.
Voice call	STT transcript from audio stream.	TTS audio response.	Later, after text engine is stable.
WhatsApp/SMS	Notification and optional patient text channel.	Patient/admin messages.	Notifications first; patient channel later.

Correct runtime sequence: Patient audio -> STT -> same text-core message API -> response text -> TTS -> patient audio. Voice does not bypass the text-core engine.

5. Low-Level Component Design

Module / component	Responsibility	Important rules
AuthModule	Login, JWT verification, current user, clinic memberships.	No endpoint except public webhooks bypasses authentication in production.
RBACGuard	Checks admin/doctor access by clinic and doctor ownership.	Doctor can see/act only on own appointments, timings, and services.
ConversationOrchestrator	Main message pipeline.	Loads session, handles active flow, saves messages, calls classifier/router/handlers.
LanguageManager	Maintains ta_tanglish or english per session.	Explicit patient language request overrides default.
IntentClassifierAdapter	LLM/mock classifier for intent and entities.	Returns JSON only. No DB writes.
ServiceRouterAdapter	Maps patient problem to clinic service profile.	Uses clinic_services only for current clinic. No phrase tables.
StateMachineRegistry	Routes active current_flow to booking/cancel/reschedule/handoff machines.	Active-flow messages use state-specific parsing before generic unknown fallback.
TemplateRenderer	Renders language-specific templates.	No random inline text in handlers except emergency fallback if template missing.
ActionValidator	Validates every write action.	Checks clinic scope, role, doctor-service-slot relation, status, idempotency.
AppointmentService	Create, confirm, cancel, reschedule, mark visited, manual booking.	Uses transactions and audit logs.
SlotService	Find, hold, release, expire, block slots.	Uses row locks and prevents double booking.
PatientVisitService	Creates visit history and finds returning-patient matches.	Visited reason is required when marking visited.
KnowledgeService	Approved Q&A search and knowledge maintenance.	Only approved answers. Structured facts never answered from KB if structured DB has source.
CallInboxService	Call list, recording metadata, transcript, summaries, outcomes.	Admin-only. Audio 10 days, transcript 30 days.
NotificationService	WhatsApp/SMS/emergency/callback notifications.	Async outbox and retry. Send patient confirmation only after clinic confirms.
AuditService	Records internal and user actions.	Old/new values, actor, clinic, source, request id.

6. Agent Design

The Vaidya agent is a controlled receptionist agent. It is not a general chatbot and not a medical advisor. It is split into small model tasks plus deterministic backend state machines.

Agent sub-part	Input	Output	Allowed to act?
Intent classifier	Patient message + session hints.	intent, entities, language, safety flags, confidence.	No
Entity extractor	Patient message + expected state.	date, time, name, doctor, problem, yes/no, slot.	No
Service router	Patient problem + active clinic service profiles.	matched clinic_service_id or unsupported/clarification.	No
Knowledge answerer	Patient question + approved knowledge search result.	approved answer or no-answer fallback.	No
Response planner	State-machine decision.	template_key + variables.	No
Backend domain service	Validated action proposal.	DB transaction and persisted state.	Yes

6.1 Agent hard boundaries

- The LLM must not run SQL, access DB clients, or call repository functions.
- The LLM must not create, confirm, cancel, reschedule, or delete appointments directly.
- The LLM must not edit doctor schedules, clinic hours, services, fees, holidays, knowledge approvals, or user permissions.
- The LLM must not give diagnosis, medicine, dosage, treatment, or severity advice.
- The LLM must not invent fees, timings, insurance rules, scan instructions, services, or available slots.
- If information is missing or unapproved, the response is clinic staff will confirm this.
- Emergency response is fixed template and backend creates emergency alert notification.

6.2 Classifier contract

Input:

```
{
  clinic_id,
  session_id,
  message_text,
  current_flow,
  current_state,
  language_style,
  known_collected_fields
}
```

Output:

```
{
  intent,
  confidence,
  language_style,
  entities: {
    patient_name,
    doctor_name,
    reason_for_visit,
    date,
    time_preference,
    visit_type,
    topic,
    requested_language_style
  },
  safety: {
    is_emergency,
    is_medical_advice_request,
    reason
  },
  needs_clarification
}
```

6.3 Agent processing order

1. Load session and acquire per-session lock.
2. Store patient message.
3. Run explicit language switch parser.
4. Run emergency and medical-advice classification or safety detector.
5. If active flow exists, route to current state machine with state-specific parsers first.
6. If no active flow, classify new intent.
7. If booking/problem requires routing, call `ServiceRouterAdapter` with clinic service profiles.
8. State machine updates `collected_json` and decides next prompt/action proposal.
9. `ActionValidator` validates any write action.
10. Domain service executes DB transaction.
11. `TemplateRenderer` creates reply in current language.
12. Store assistant message, audit events, and return response.

7. Booking Flow Low-Level Design

Booking is the highest-risk flow. It must support compound answers, returning-patient history, service routing, side questions, slot holding, patient confirmation, and pending/confirmed status based on admin config.

7.1 Booking state machine

BOOKING_STARTED
ASK_PROBLEM_OR_DOCTOR
EXTRACT_BOOKING_DETAILS
CHECK_RETURNING_PATIENT_HISTORY
ROUTE_SERVICE
SELECT_DOCTOR
ASK_DATE
ASK_TIME
CHECK_SLOTS
PROPOSE_SLOTS
HOLD_SLOT
ASK_PATIENT_NAME
ASK_PHONE_IF_NEEDED
CONFIRM_DETAILS
CREATE_APPOINTMENT_REQUEST
DONE

Required field	Source	Can infer?	Required before action?
patient_phone	Caller ID/session or user input.	Yes, from call/session.	Yes
patient_name	User input or existing patient match.	Maybe.	Yes
reason_for_visit	User message.	No.	Yes - must be stored on appointment.
clinic_service_id	Service router or doctor selection.	Yes after routing.	Yes
doctor_id	Previous visit, doctor name, or doctor_services mapping.	Yes.	Yes
date	User message.	No.	Yes
time_preference or slot_id	User message/slot selection.	No.	Yes
confirmation	User says yes/seri/okay.	No.	Yes

7.2 Booking decision order

13. Greeting asks what problem/appointment the patient needs.
14. Extract all provided details from the first response: problem, doctor, date, time, name, follow-up wording.
15. If phone/name matches an existing patient, fetch patient history and same-service/same-problem visits.
16. If same service/problem follow-up exists and previous doctor is valid, prefer previous doctor.
17. If no same-problem follow-up, route problem to clinic_services using service profiles.
18. If clinic does not handle the service, respond unsupported and do not book a wrong doctor.
19. Find doctor(s) from doctor_services. If multiple doctors match, ask patient to choose.
20. Ask only missing date/time/name/phone questions.
21. Check available slots by clinic_id + doctor_id + clinic_service_id + date/time.
22. Hold selected slot with DB transaction and short expiry.
23. Ask final confirmation.
24. Create appointment as pending_confirmation by default or confirmed if admin enabled auto-confirm.

7.3 Compound message examples

Patient message	Extracted fields	Next action
Naalaikku evening knee pain-ku appointment venum. En peyar Kumar.	date=tomorrow, time=evening, reason=knee pain, patient_name=Kumar.	Check history, route service, propose slots.
Dr Priya May 16 evening appointment. Name Meena.	doctor=Dr. Priya, date=2026-05-16, time=evening, patient_name=Meena.	Check Dr. Priya slots. Do not ask name/date/time again.
Knee pain follow-up. Kumar. Saturday morning.	reason=knee pain, visit_type=follow_up, name=Kumar, date=Saturday, time=morning.	Find previous knee visit and prefer previous doctor if valid.
Dr Priya fees evlo?	side question ask_fee, doctor_name=Dr.	Answer fee and resume booking, asking if

	Priya.	Dr. Priya should be selected.
Vendam.	negative/cancel phrase.	Stop active booking flow and release held slot.

7.4 Returning-patient logic

```

findReturningPatientMatch(clinic_id, patient_phone, patient_name?, current_service_id, normalized_reason):
  candidates = patients where clinic_id and phone match
  if multiple patient names share phone and patient_name is unknown:
    ask who the appointment is for
  find latest patient_visits where:
    clinic_id matches
    patient_id matches
    clinic_service_id matches current service
    normalized_reason similar if available
    followup_valid_until >= now OR clinic setting allows previous-doctor preference
  if found and doctor is active and still handles service:
    preferred_doctor_id = previous_visit.doctor_id
    routing_source = returning_patient_followup
  else:
    routing_source = service_router

```

Scenario	Behavior
Existing patient same problem	Prefer previous doctor and show history in booking panel.
Existing patient different problem	Use service router for new problem; history remains visible to admin.
Same phone multiple patients	Ask which patient appointment is for before using history.
Previous doctor inactive or no slots	Explain previous doctor unavailable and offer another doctor mapped to service if configured.
No history	Normal service routing.

8. Cancel, Reschedule, Handoff, FAQ, and Emergency Flows

8.1 Cancel flow

CANCEL_STARTED -> FIND_EXISTING_APPOINTMENT -> SELECT_APPOINTMENT_IF_MULTIPLE -> CONFIRM_CANCEL -> CANCEL_APPOINTMENT -> DONE

- Find upcoming appointments by clinic_id + patient_phone + status in pending_confirmation/confirmed.
- If multiple appointments exist, show numbered list and ask which one.
- Always ask confirmation before cancelling.
- On confirmation, update appointment status to cancelled and release associated slot.
- On vendam/no, do not cancel and end flow safely.
- Notify admin if patient cancelled by call and link to Call inbox.

8.2 Reschedule flow

RESCHEDULE_STARTED -> FIND_EXISTING_APPOINTMENT -> SELECT_APPOINTMENT_IF_MULTIPLE -> ASK_NEW_DATE -> ASK_NEW_TIME -> CHECK_NEW_SLOTS -> PROPOSE_NEW_SLOTS -> CONFIRM_RESCHEDULE -> UPDATE_APPOINTMENT -> DONE

- Patient can request reschedule by call/text. Admin/doctor threshold applies only to manual UI edits.
- Default keeps same doctor and service unless patient explicitly asks otherwise.
- Hold new slot before confirmation.
- On final confirmation, release old slot and mark new slot pending/confirmed based on booking mode.
- On negative confirmation, release new held slot and keep old appointment unchanged.
- Notify admin and show request in Appointments -> Reschedule / cancel section.

8.3 Handoff/callback flow

HANDOFF_STARTED -> ASK_REASON_OPTIONAL -> ASK_NAME_IF_NEEDED -> ASK_PHONE_IF_NEEDED -> CREATE_CALLBACK_REQUEST -> DONE

- Triggered when patient asks for receptionist/staff/human/callback or when Vaidya cannot answer safely.
- Collect reason, name if missing, and phone if not available from session.
- Create callback_request with status pending.
- If user says vendam/no before completion, no callback request is created.

8.4 FAQ/knowledge flow

- Structured facts such as doctor fees, clinic hours, doctor schedules, location, services, and holidays come from structured tables, not knowledge base.
- Knowledge base is only for clinic-approved Q&A: scan preparation, parking, first visit documents, insurance explanation, report process, etc.
- Only status=approved answers can be used.
- If no approved answer exists, reply: Clinic staff will confirm this.
- If asked during booking, answer the side question and resume current booking prompt without resetting flow.

8.5 Emergency and medical-advice behavior

Case	Behavior	Action
Emergency	Instruct patient to call 108 / nearest hospital.	Create emergency_incident and clinic alert notification.
Medical advice request	Refuse medical advice and offer appointment/callback.	Do not answer from knowledge base.
Emergency during active flow	Emergency overrides booking/cancel/reschedule/handoff.	Stop unsafe continuation and alert clinic.
Medication/dosage question	Never suggest medicine, dosage, treatment, or seriousness.	Use fixed medical advice refusal template.

9. Application Design: Five Tabs

The application has five tabs. Home is useful but lowest priority. Appointment operations, clinic setup, knowledge base, and call inbox drive production value. The clinic-facing UI should not show internal AI metrics such as model confidence or vector score.

Tab	Admin view	Doctor view	Priority
Home	Clinic summary, pending actions, callbacks, emergency alerts, agent status.	Own upcoming appointments and own pending actions only.	Low after core workflows.
Call inbox	Visible. Last calls, preview, recording, transcript, outcome.	Not visible.	High after voice integration.
Appointments	All pending/confirmed/visited/reschedule/cancel records.	Own appointments and own visited records only.	High.
Clinic setup	Full setup access.	Own timings and own services only.	High.
Knowledge base	Upload/use template, add/edit/approve Q&A.	No access initially.	High.

9.1 Home

- Cards: calls today, pending confirmations, callbacks, emergency alerts, no-answer knowledge gaps.
- Agent status: active/off/after-hours/overflow mode.
- Action list: pending appointments, callbacks, emergency alerts.
- Doctor view only shows own appointments and own visit-related items.
- Lowest build priority; must not block core rollout.

9.2 Call inbox - admin only

Section	Fields / actions
Call list	Call time, phone, patient name if known, duration, summary, outcome, action needed. Filter by date/outcome.
Call detail preview	Intent, outcome, appointment/callback/emergency link, patient info, summary, service/doctor if any.
Recording + transcript	Audio player for 10 days, transcript for 30 days, speaker turns, download access if allowed.

Correction: call recordings are not available for 30 days. Audio is 10 days. Transcript is 30 days.

9.3 Appointments

Section	Visible to	Behavior
Pending confirmation	Admin; doctors for own appointments if allowed.	Shown when booking_mode=pending_confirmation. Confirm, edit time, call, cancel.
Confirmed	Admin all; doctor own only.	View confirmed appointments. Doctor can mark own patient visited if enabled.
Visited	Admin all; doctor own only.	Receptionist/doctor marks visited and must enter visit reason. Creates patient_visits row.
Reschedule / cancel	Admin only.	Lists patient requested cancel/reschedule actions from calls. Admin takes final action.

9.4 Clinic setup

Section	Admin permissions	Doctor permissions
Bot answering mode	Edit agent on/off, answering mode, fallback phone, booking mode, concurrency, retention.	No access.
Doctor available timings	Edit all doctors. Conflict check required.	Edit own timings only.
Clinic regular working hours	Edit clinic hours.	Read-only or hidden.
Holiday setup	Add/edit holidays. Conflict check required.	Read-only or hidden.
Services Vaidya can route	Edit doctor-service mappings, fees, handled/non-handled conditions.	Edit own services only if allowed.

Correction: adding a holiday or changing doctor timings must never auto-cancel appointments. The system must detect conflicts, show the conflict list, and require admin manual action.

9.5 Knowledge base

- Use the Q&A template directly inside the app; download/upload is optional.
- Upload DOCX parses Q&A rows into pending_review.
- Manual Q&A adds one question and answer row.
- Bulk and single approval/edit actions.
- Approved answers can be edited later; old version is archived.
- Knowledge base must not ask for clinic details, doctor schedules, fees, service routing, or holidays; those come from structured setup UI.

10. Role-Based Access Control

Capability	Admin	Doctor
Login	Yes	Yes
Home	Full clinic summary.	Own appointment summary.
Call inbox	Yes.	No.
View appointments	All clinic appointments.	Own appointments only.
Approve pending appointment	All.	Own appointments only.
Manual appointment	Any doctor; outside hours with explicit override.	Own slots only; no outside-hours override unless later enabled.
Edit appointment time	All, subject to threshold/override.	Own appointments, subject to threshold.
Mark visited	All.	Own only. Visit reason required.
Edit doctor timings	All doctors.	Own only.
Edit services/fees	All.	Own services only if allowed.
Bot on/off	Yes.	No.
Knowledge approve/edit	Yes.	No initially.
Emergency alert view	Yes.	No unless later mapped to own appointment.

RBAC guard pattern:

1. Verify authenticated user.
2. Verify user belongs to clinic.
3. Load clinic role from clinic_users.
4. If role=doctor, enforce doctor_id ownership on appointment/timing/service APIs.
5. If role=admin, allow clinic-scoped action.
6. Write audit log for create/update/cancel/override actions.

11. API Design

All APIs are clinic-scoped and protected by authentication except public voice webhooks. Production responses must not expose debug fields. Each write endpoint accepts an idempotency_key where duplicate execution is possible.

11.1 Auth and profile APIs

Method	Endpoint	Roles	Purpose
POST	/v1/auth/login	Public	Login with provider/email/phone.
POST	/v1/auth/logout	User	End session.
GET	/v1/me	User	Return user, clinic memberships, roles, permissions.

11.2 Conversation/agent APIs

Method	Endpoint	Purpose
POST	/v1/conversations	Create text/voice conversation session.
POST	/v1/conversations/{sessionId}/messages	Send patient text to text-core engine. Used by text console and voice adapter after STT.
GET	/v1/conversations/{sessionId}	Return session and transcript for admin/debug access.
POST	/v1/voice/calls/webhook	Telephony webhook entry point later.
POST	/v1/voice/calls/{callId}/events	Voice event updates: started, ended, transcript, recording ready.

11.3 Appointment APIs

Method	Endpoint	Roles	Purpose
GET	/v1/appointments	Admin/Doctor	List appointments. Doctor sees own only.
GET	/v1/appointments/{id}	Admin/Doctor-own	Appointment detail with history if allowed.
POST	/v1/appointments/manual	Admin/Doctor-own	Create manual appointment. Admin can override outside hours.
POST	/v1/appointments/{id}/confirm	Admin/Doctor-own	Confirm pending appointment and enqueue patient notification.
POST	/v1/appointments/{id}/cancel	Admin/Doctor-own	Cancel appointment, release slot, enqueue notification.
POST	/v1/appointments/{id}/reschedule	Admin/Doctor-own	Reschedule appointment with transaction.
POST	/v1/appointments/{id}/mark-visited	Admin/Doctor-own	Mark visited and create patient_visit with reason.
GET	/v1/appointments/actions/reschedule-cancel	Admin	List user-requested reschedule/cancel items.

11.4 Clinic setup APIs

Method	Endpoint	Roles	Purpose
GET/PATCH	/v1/clinic/settings	Admin	Agent mode, fallback phone, booking mode, concurrency, retention.
GET/POST/PATCH	/v1/doctors	Admin	Manage doctors.
GET/PATCH	/v1/doctors/{id}/schedules	Admin/Doctor-own	Manage doctor timings with conflict check.
GET/PATCH	/v1/clinic-hours	Admin	Manage clinic working hours.
GET/POST/PATCH	/v1/holidays	Admin	Manage holidays with conflict check.
GET/POST/PATCH	/v1/clinic-services	Admin/Doctor-own limited	Manage service profiles and mappings.

11.5 Knowledge APIs

Method	Endpoint	Roles	Purpose
GET	/v1/knowledge	Admin	List Q&A rows by status/category.
POST	/v1/knowledge/manual	Admin	Add manual Q&A. Default pending_review.
POST	/v1/knowledge/upload-docx	Admin	Parse DOCX Q&A into pending_review rows.
POST	/v1/knowledge/{id}/approve	Admin	Approve answer for runtime use.
POST	/v1/knowledge/bulk-approve	Admin	Bulk approve selected Q&A.
PATCH	/v1/knowledge/{id}	Admin	Edit Q&A; archive old version if approved.
POST	/v1/knowledge/{id}/disable	Admin	Disable answer; no hard delete.

11.6 Call inbox and notification APIs

Method	Endpoint	Roles	Purpose
GET	/v1/calls	Admin	List calls by date/outcome/action status.
GET	/v1/calls/{id}	Admin	Call details.
GET	/v1/calls/{id}/recording-url	Admin	Short-lived signed URL if audio not expired.
GET	/v1/calls/{id}/transcript	Admin	Transcript if within retention.
GET	/v1/notifications/events	Admin	Notification history and status.
POST	/v1/notifications/test	Admin	Send test notification.

12. Database Low-Level Design

Every clinic-owned table must include `clinic_id`. Database constraints must protect against clinic/doctor/service mismatches even if application code has a bug.

12.1 Core tables

Table	Purpose	Important fields
clinics	Clinic master record.	id, name, phone, address, timezone, default_language, active.
clinic_settings	Agent and booking behavior.	clinic_id, agent_enabled, answering_mode, fallback_phone, booking_mode, max_concurrent_calls, recording_retention_days, transcript_retention_days.
users	Login identity.	id, name, email, phone, auth_provider_id, active.
clinic_users	Clinic role membership.	clinic_id, user_id, role, doctor_id nullable, active.
doctors	Doctor profile, login optional.	id, clinic_id, user_id nullable, name, qualification, active.
clinic_services	Service profile Vaidya can route.	id, clinic_id, service_key, name, handles_json, does_not_handle_json, red_flags_json, routing_examples_json, active.
doctor_services	Many-to-many doctor-service mapping.	clinic_id, doctor_id, clinic_service_id, active, fee fields or fee rule link.
doctor_schedules	Doctor available timings.	clinic_id, doctor_id, day_of_week, start_time, end_time, active, effective_from/to.
clinic_hours	Regular clinic hours.	clinic_id, day_of_week, open_time, close_time, active.
clinic_holidays	Holiday / closure setup.	clinic_id, holiday_date, reason, full_day, start_time/end_time.

12.2 Appointment and patient tables

Table	Purpose	Important fields
patients	Patient identity within clinic.	id, clinic_id, name, phone, preferred_language, active. Avoid unique phone-only because families share phones.
patient_visits	Completed visit history for follow-up routing.	clinic_id, patient_id, doctor_id, clinic_service_id, appointment_request_id, reason_for_visit, normalized_reason, visited_at, followup_valid_until.
appointment_slots	Bookable doctor-service time windows.	clinic_id, doctor_id, clinic_service_id, start_time, end_time, status, held_by_session_id, hold_expires_at.
appointment_requests	Appointment lifecycle.	clinic_id, patient_id nullable, doctor_id, clinic_service_id, slot_id, patient_name, patient_phone, reason_for_visit, status, is_followup, routing_source.
appointment_events	Timeline of appointment changes.	appointment_id, event_type, actor_user_id, source, old_json, new_json, created_at.

12.3 Conversation, call, knowledge, notification tables

Table	Purpose	Important fields
conversation_sessions	Text/voice conversation state.	clinic_id, patient_phone, current_flow, current_state, language_style, collected_json, status.
conversation_messages	Visible patient/assistant transcript.	session_id, clinic_id, sender, message_text, intent, state_before/after, reply_template_key.
message_idempotency_keys	Duplicate message/action prevention.	clinic_id, session_id, idempotency_key, response_json.

calls	Voice call metadata.	clinic_id, session_id, phone, started_at, ended_at, duration, outcome, recording_url, recording_expires_at, summary.
call_transcripts	Speaker turns from voice calls.	clinic_id, call_id, speaker, text, started_at_ms, ended_at_ms, expires_at.
clinic_knowledge_base	Approved Q&A source.	clinic_id, question, answer, category, status, source_file, source_page, search_text, embedding, approved_at.
knowledge_answer_versions	Version history for edited approved Q&A.	knowledge_id, old_answer, new_answer, edited_by, edited_at.
callback_requests	Staff callback tasks.	clinic_id, patient_name, patient_phone, reason, status, source_session_id.
emergency_incidents	Emergency call events.	clinic_id, patient_phone, patient_message, status, source_call_id, source_session_id.
notification_events	Async notification outbox.	clinic_id, type, recipient, channel, payload_json, status, attempts, next_retry_at.
audit_logs	Internal and user action logs.	clinic_id, user_id, session_id, event_type, old_json, new_json, created_at.

12.4 Status values

Entity	Statuses
appointment_requests	pending_confirmation, confirmed, visited, cancelled, rescheduled, no_show, callback_needed, conflict_required
appointment_slots	available, held, pending_confirmation, confirmed, blocked, cancelled
clinic_knowledge_base	pending_review, approved, disabled, needs_update
callback_requests	pending, in_progress, completed, cancelled
notification_events	pending, sending, sent, failed, dead_letter

13. Action Safety and False-Entry Prevention

The production system must assume the LLM can misunderstand. All writes are protected by ActionValidator and domain services. The LLM cannot create false entries because it has no write capability.

13.1 ActionValidator checks

Action	Required checks
Create appointment	Authenticated/session source valid; clinic scope valid; doctor belongs to clinic; service belongs to clinic; doctor handles service; slot belongs to same clinic/doctor/service; slot is available or held by same session; patient confirmed; idempotency key unused.
Confirm appointment	Admin or owning doctor; appointment belongs to clinic; status pending_confirmation; slot still linked; create notification_event after commit.
Cancel appointment	Admin or owning doctor; appointment belongs to clinic; cancellable status; release slot; no hard delete; audit event.
Reschedule appointment	Old appointment belongs to clinic; new slot belongs to same clinic; new slot held; update appointment and slot atomically; release old slot after success.
Manual appointment	Admin can override outside hours with explicit reason; doctor can create only own slots; no double booking; audit override.
Mark visited	Admin or owning doctor; appointment confirmed; visit reason required; creates patient_visits row.
Edit doctor timing/holiday	Admin or owning doctor where allowed; conflict detection required; do not auto-cancel existing appointments.
Approve knowledge	Admin only; status moves to approved; versioning and audit log.
Agent on/off	Admin only; fallback phone must exist if off/forwarding mode.

13.2 Slot locking transaction

```
BEGIN;

SELECT *
FROM appointment_slots
WHERE id = $slot_id
      AND clinic_id = $clinic_id
      AND doctor_id = $doctor_id
      AND clinic_service_id = $clinic_service_id
      AND status = 'available'
FOR UPDATE;

UPDATE appointment_slots
SET status = 'held',
    held_by_session_id = $session_id,
    hold_expires_at = NOW() + INTERVAL '5 minutes',
    updated_at = NOW()
WHERE id = $slot_id;

COMMIT;
```

13.3 No hard delete policy

- Appointments are cancelled, rescheduled, or marked no_show/visited. They are not deleted.
- Knowledge answers are disabled or archived, not deleted.
- Doctors/services are made inactive, not deleted, if historical appointments exist.
- Audit logs are append-only.

14. Notifications, Recordings, Transcripts, and Retention

14.1 Notification rules

Event	Patient notification?	Clinic notification?	Notes
Appointment pending_confirmation created	No	Optional staff action notification	Clinic must confirm first.
Appointment confirmed by admin/doctor	Yes	Audit only	WhatsApp/SMS goes out only after confirmed.
Appointment time edited by admin/doctor	Yes	Audit only	Notify changed date/time.
Appointment cancelled by admin/doctor	Yes	Audit only	Notify cancellation.
Patient cancels by call	No unless clinic confirms cancellation messaging policy	Yes	Show in reschedule/cancel and call inbox.
Patient requests reschedule by call	No final patient notification yet	Yes	Admin takes final action.
Callback request created	No or optional acknowledgement	Yes	Staff action.
Emergency detected	Emergency instruction spoken immediately	Yes urgent alert	Create emergency_incident.

14.2 Notification outbox

Domain transaction:

```
update appointment status to confirmed
insert notification_events row with status='pending'
commit
```

Worker:

```
fetch pending events
call WhatsApp/SMS provider
update status=sent or failed
retry failed events with backoff
move to dead_letter after max attempts
```

14.3 Retention

Data	Retention	Storage
MP3/audio recording	10 days per clinic.	Object storage. DB stores recording_url and expires_at.
Transcript text	30 days.	Postgres call_transcripts.
Call summary/metadata	Configurable, default 180 days or longer.	Postgres calls table.
Appointments and visits	Operational history.	Postgres.
Audit logs	Configurable retention, default 1-3 years for production.	Postgres/archive later.

15. Voice Integration Design

Voice is added after text QA is stable. The same ConversationOrchestrator and state machines are used. Voice adapter converts audio to text and text to audio, but does not own business logic.

Patient call

- > TelephonyProvider receives call / media stream
- > VoiceSessionManager creates call + conversation_session
- > STTProvider converts speech to text
- > POST message to ConversationOrchestrator internally
- > response text from TemplateRenderer
- > TTSPProvider converts to audio
- > TelephonyProvider plays audio
- > call ends; call summary, recording, transcript stored

Adapter	Purpose	Provider strategy
TelephonyProvider	Call answer, forward, stream, recording, hangup.	Exotel/Twilio implementations behind same interface. Do not hardcode one.
STTProvider	Speech-to-text for Tamil/Tanglish/English.	Sarvam STT implementation first; fallback provider later.
TTSPProvider	Text-to-speech response.	Sarvam Bulbul implementation first; cache common prompts.
VoiceSessionManager	Maps telephony call ID to conversation session and call record.	Handles call start/end/error events.

16. QA, Production Deployment, Observability

Environment	Purpose	Data
Local	Developer work, unit tests, text console.	Local Postgres or QA DB with fake data.
QA	Shared integration testing before production.	Supabase Free/Pro QA project; fake clinic data; no real patient data unless approved.
Production	Pilot and paid clinics.	Managed Postgres, backups, object storage, Redis/queue, monitoring.

16.1 Observability

- Structured logs with request_id, clinic_id, user_id/session_id, action, latency.
- Error tracking with Sentry or equivalent.
- Metrics: API latency, DB latency, queue backlog, notification failure rate, LLM latency, STT/TTS latency, no-answer knowledge count, slot conflict count.
- Audit logs are separate from conversation messages. Conversation messages are transcript. Audit logs are internal system events.
- Alert on failed notification queue, DB connection errors, voice provider errors, high LLM failure rate, emergency incidents, and expired slot cleanup failures.

16.2 Capacity plan

Stage	Clinics	Concurrent calls per clinic	Notes
QA/demo	1-3 fake/demo clinics	Text only or 1-2 test calls	Supabase Free can be enough.
Pilot production	1-5 clinics	Starter 1, Pro 3, Clinic+ 5	Use Supabase Pro or managed Postgres. Keep pending confirmation default.
Early paid production	5-20 clinics	Plan-based limits	Monitor DB, provider limits, recordings, queue. Upgrade compute as needed.
Scale phase	20+ clinics	Custom for high-volume clinics	Move to larger DB compute, separate workers, provider rate-limit planning.

Do not promise unlimited parallel calls. Product should have plan-based concurrency: Starter 1, Pro 3, Clinic+ 5, custom after load testing.

17. Implementation Phases

Phase	Scope	Exit criteria
1. Foundation	NestJS app, auth, RBAC, DB migrations, audit logs, clinic_users.	Admin/doctor login works; role guards enforce ownership.
2. Appointments	Slots, manual booking, confirm/cancel/reschedule, mark visited, patient visits.	No double booking; doctor own-appointment restrictions pass tests.
3. Clinic setup	Agent mode, fallback phone, doctor timings, clinic hours, holidays, services.	Conflict detection works; no auto-cancel.
4. Text agent core	Conversation sessions, classifier, service router, booking flow, side questions.	Golden text flows pass.
5. Knowledge base	Manual Q&A, DOCX upload, approval, approved-only runtime answers.	Pending/unapproved answers never used; tenant isolation tested.
6. Notifications	Notification outbox, WhatsApp/SMS provider, retries, delivery status.	Patient notifications only after confirmed; edit/cancel notifications work.
7. Call inbox	Calls, summaries, recordings metadata, transcripts, retention cleanup.	Admin-only access; audio 10 days, transcript 30 days.
8. Voice adapter	Telephony/STT/TTS integration, call lifecycle, recording.	Internal phone tests pass before any patient calls.
9. Pilot production	1 clinic after-hours/overflow mode.	Daily review, no safety failures, measurable appointment capture.

18. Corrections Applied From Design Review

Issue found in earlier designs	Final correction
FastAPI + Zod mixed backend terminology.	Final backend is NestJS + TypeScript + Zod. If FastAPI were used, it would use Pydantic, but this final plan does not use FastAPI.
Twilio hardcoded as provider.	Use provider adapter. Exotel and Twilio are implementations. India pilot likely starts with Exotel if best operationally.
Sarvam-M (30B) naming.	Use configurable model IDs: sarvam-30b primary, sarvam-105b fallback.
Simple appointment/slot statuses.	Use full lifecycle statuses for pending, confirmed, visited, cancelled, rescheduled, conflict, held, blocked.
Users table with role only.	Use users + clinic_users + doctors. Doctor login maps to doctor_id; doctor profile can exist without login.
Missing doctor_services many-to-many.	Add doctor_services. One doctor can handle many services; one service can have many doctors.
Loose service conditions text.	Use structured service profile: handles, does_not_handle, red_flags, routing_examples, fees, active.
Knowledge base used for fees/timings.	Structured facts come from structured DB; KB is approved Q&A only.
Holiday/timing change auto-cancels appointments.	Reject. Show conflicts and require admin manual action.
Recording shown as 30 days in UI.	Audio 10 days, transcript 30 days.
Routing quality/internal AI screens.	Do not show to clinic users. Keep internal debug/admin only.
Direct previous-doctor routing.	Prefer previous doctor only after validating patient identity, service/problem match, doctor active, and slots.
No notification retry design.	Use notification_events outbox and worker retries.

19. Production Readiness Checklist

- LLM has no DB credentials and cannot call write actions.
- All writes pass through ActionValidator and domain services.
- Every clinic-owned query filters clinic_id.
- Doctor role can access only own appointments/timings/services.
- Call inbox is admin only.
- Appointment reason/problem is stored for every booking.
- Mark visited requires visit reason and creates patient_visits row.
- Patient history is shown during admin appointment booking/confirmation.
- Slot hold uses transaction and row lock.
- Idempotency prevents duplicate appointments/callbacks/notifications.
- Notification is sent to patient only after appointment confirmation.
- Emergency creates emergency_incident and clinic alert.
- Medical advice is refused with fixed template.
- Knowledge uses approved Q&A only; missing answer falls back safely.
- Audio recording cleanup deletes after 10 days.
- Transcript cleanup deletes after 30 days.
- Holiday/timing changes show conflicts and do not auto-cancel.
- Production DB has automated backups and monitored connection pool.
- QA and production are separate environments.
- Voice provider, STT, TTS, LLM are adapter-based and configurable.

20. Final Build Recommendation

Build the final product from scratch using this architecture: Next.js frontend, NestJS + Fastify backend, PostgreSQL + Drizzle + SQL migrations, Supabase Auth or equivalent JWT provider, BullMQ + Redis/Valkey, object storage for recordings, provider adapters for voice/STT/TTS/LLM, and a modular monolith domain architecture.

Use the shorter implementation plan as an executive roadmap, but use this low-level design as the engineering blueprint. The LLD contains the details needed to avoid wrong bookings, false DB entries, missed permissions, unsafe medical responses, and production data issues.

The document is production-ready as a design baseline. Before coding each phase, convert the relevant sections into implementation tickets, API contracts, and acceptance tests.

27. Integrated Final Corrections and Best Decisions

This section integrates the final production corrections, best recommendations, and best decisions from the review of the high-level implementation plan, the low-level design, and the clinic UI direction. This document is now the single engineering source of truth. The high-level implementation plan can be used for executive alignment, but implementation agents must follow this document for API, database, state-machine, safety, UI, and production behavior.

Area	Best decision	Why this is best for Vaidya
Backend	NestJS + TypeScript + Fastify adapter.	Best fit for a fresh-start modular monolith, shared frontend/backend types, RBAC guards, WebSocket readiness, queues, and maintainability for a small team. FastAPI is good but would force a second language stack.
Frontend	Next.js + TypeScript + Tailwind.	Role-based admin/doctor portal, production routing, reusable components, and deployment readiness.
Database	PostgreSQL with SQL-first migrations and Drizzle.	Required for transactions, row locks, composite foreign keys, status constraints, exclusion constraints, and future pgvector.
Telephony/messaging	Provider adapter pattern, not Twilio-only or Exotel-only.	India cost/compliance/support may favor Exotel, while Twilio may be useful in some contexts. Keep provider swappable.
LLM	sarvam-30b primary, sarvam-105b fallback, configurable adapters.	Corrects old Sarvam-M naming and avoids hardcoding one model. Use fallback only when confidence is low or JSON fails.
Voice	Voice added after text QA. Voice is only channel adapter.	Prevents debugging STT/TTS while core booking and safety logic are still unstable.
Knowledge	Q&A only. Structured operational facts stay in structured DB.	Prevents stale PDF/FAQ answers from overriding current doctor fees, schedules, hours, locations, holidays, or services.
UI	Five clinic tabs: Home, Call Inbox, Appointments, Clinic Setup, Knowledge Base.	Matches doctor/receptionist workflow. No separate Safety tab or Routing Quality screen for clinic users.
Holiday/timing conflicts	Detect conflicts and block/save with review. Never auto-cancel silently.	Auto-cancel is unsafe. Staff must choose cancel/reschedule and notify patient after action.
Record retention	Audio 10 days, transcript 30 days.	Balances clinic review needs, privacy, and storage cost.
Booking default	pending_confirmation by default; auto-confirm only by admin config.	Keeps clinic in control during pilot and reduces false-confirmation risk.
Patient cancel/reschedule by call	Create action request for admin by default; optional auto-cancel can be added later by config.	Safer for speech/LLM misunderstandings and matches the Appointments Reschedule/Cancel section.

28. API Contract Addendum

All coding agents must implement APIs with stable request/response contracts. Field names must be snake_case over the wire. Backend code may use camelCase internally. Every mutation must include an idempotency_key unless the endpoint is explicitly read-only.

28.1 Standard response and error format

Success response example:

```
{
  "data": { ... },
  "meta": {
    "request_id": "req...",
    "debug": null
  }
}
```

Error response example:

```
{
  "error": {
    "code": "SLOT_NOT_AVAILABLE",
    "message": "Selected slot is no longer available.",
    "details": {},
    "request_id": "req..."
  }
}
```

Error code	When used	HTTP
UNAUTHORIZED	Missing/invalid login token.	401
FORBIDDEN	User belongs to clinic but lacks permission.	403
CLINIC_NOT_FOUND	Clinic does not exist or inactive.	404
DOCTOR_NOT_OWNER	Doctor attempted to access another doctor record.	403
VALIDATION_ERROR	Request body or params invalid.	400
SLOT_NOT_AVAILABLE	Slot is no longer available or is not for doctor/service.	409
SLOT_HOLD_EXPIRED	Held slot expired before confirmation.	409
APPOINTMENT_NOT_CONFIRMABLE	Appointment status cannot be confirmed.	409
APPOINTMENT_NOT_CANCELLABLE	Appointment status cannot be cancelled.	409
CONFLICTING_APPOINTMENTS	Schedule/holiday change conflicts with existing appointments.	409
IDEMPOTENCY_CONFLICT	Same idempotency key used with different payload.	409
KNOWLEDGE_NOT_APPROVED	Attempt to use unapproved knowledge.	409
PROVIDER_FAILURE	External provider failed after retries or timeout.	502

28.2 Conversation APIs

Endpoint	Request	Response	Notes
POST /v1/conversations	{ clinic_id, channel, patient_phone?, language_style? }	{ session_id, status, current_flow, current_state, language_style }	Creates text/test/voice conversation session. Validates clinic active.
POST /v1/conversations/{session_id}/messages	{ message_text, idempotency_key }	{ reply_text, current_flow, current_state, actions[], debug? }	Acquires session lock. Stores patient and assistant messages. Does not expose debug unless DEBUG_API=true and role permits.
GET /v1/conversations/{session_id}	Path only	{ session, messages[] }	Admin can view clinic sessions. Doctor access only if explicitly implemented later; default no call inbox.

28.3 Appointment APIs

Endpoint	Roles	Request fields	Important behavior
GET /v1/appointments	admin all; doctor own only	filters: status, doctor_id, date_from, date_to, source, patient_phone	Must scope by clinic_id. Doctor filter is forced to own doctor_id.
POST /v1/appointments/manual	admin; doctor own slots only	doctor_id, clinic_service_id, patient_name, patient_phone, reason_for_visit, slot_id or start/end, override_outside_hours?, override_reason?, idempotency_key	Admin can override outside clinic hours with reason. Doctor cannot override outside hours in v1. No one can override double booking.
POST /v1/appointments/{id}/confirm	admin; owning doctor	idempotency_key, note?	Status must be pending_confirmation. Creates notification_event for patient confirmation after commit.
POST /v1/appointments/{id}/cancel	admin; owning doctor	idempotency_key, cancellation_reason, notify_patient=true	Sets status cancelled, releases slot, writes appointment_event and notification_event.
POST /v1/appointments/{id}/reschedule	admin; owning doctor	new_slot_id or new_start/end, reason, idempotency_key	Transactional: hold new slot, update appointment, release old slot, event log, notification event.

POST /v1/appointments/{id}/mark-visited	admin; owning doctor	visit_reason, idempotency_key	visit_reason required. Creates patient_visits row. Status becomes visited.
GET /v1/appointments/{id}/history	admin; owning doctor	Path only	Returns patient history, appointment events, prior visits, and call links.

28.4 Appointment action request APIs

By default, patient-initiated cancel/reschedule through Vaidya creates an admin action request instead of instantly modifying confirmed appointments. This is safer for voice misunderstandings and aligns with the Reschedule/Cancel section in the UI.

Endpoint	Roles	Purpose
GET /v1/appointment-action-requests	admin only	Lists pending patient cancel/reschedule requests.
POST /v1/appointment-action-requests/{id}/approve	admin only	Applies requested cancel/reschedule after validation.
POST /v1/appointment-action-requests/{id}/reject	admin only	Rejects action request with reason; optionally notify patient.

28.5 Clinic setup APIs

Endpoint	Roles	Request/notes
GET/PATCH /v1/clinic/settings	admin only	agent_enabled, answering_mode, fallback_phone_number, booking_mode, max_concurrent_calls, recording_enabled, retention_days.
GET/PATCH /v1/clinic-hours	admin only	Structured clinic opening hours. Conflict check not needed unless closing hours affects existing slots.
GET/POST/PATCH /v1/holidays	admin only	Adding/editing holidays must check conflicting appointments and return CONFLICTING_APPOINTMENTS. No auto-cancel.
GET/POST/PATCH /v1/doctors	admin full; doctor own profile limited	Doctor login is optional. doctor.user_id can be null.
GET/PATCH /v1/doctors/{id}/schedules	admin all; doctor own only	Changing schedules must detect appointment conflicts. No auto-cancel.
GET/POST/PATCH /v1/clinic-services	admin all; doctor own service limited if enabled	Service profiles contain handles/does_not_handle/red_flags/routing_examples.
POST/PATCH /v1/doctor-services	admin only initially	Many-to-many doctor-service mapping.

28.6 Knowledge APIs

Endpoint	Roles	Behavior
GET /v1/knowledge	admin only	List Q&A rows by status/category/source.
POST /v1/knowledge/manual	admin only	Create manual Q&A. Default pending_review unless admin explicitly approves.
POST /v1/knowledge/upload-docx	admin only	Parse clinic Q&A template into pending_review rows. Do not store schedules/fees/hours in KB.
POST /v1/knowledge/{id}/approve	admin only	Sets status approved, approved_by, approved_at, creates version/audit.
POST /v1/knowledge/bulk-approve	admin only	Bulk approve reviewed rows.
PATCH /v1/knowledge/{id}	admin only	Edits answer; approved rows create new version and regenerate embedding later.
POST /v1/knowledge/{id}/disable	admin only	Disables answer; no hard delete in normal UI.

28.7 Call inbox and notification APIs

Endpoint	Roles	Behavior
GET /v1/calls	admin only	List calls by date/outcome/action_needed.
GET /v1/calls/{id}	admin only	Call summary, outcome, linked appointment/callback/emergency.
GET /v1/calls/{id}/transcript	admin only	Transcript available for 30 days.
GET /v1/calls/{id}/recording-url	admin only	Returns short-lived signed URL if audio still within 10-day retention.
GET /v1/notification-events	admin/platform support	View delivery status, retries, provider errors.
POST /v1/notifications/test	admin only	Send test notification to configured clinic contact.

29. Database Constraints, Indexes, and State Transitions

Database constraints are part of the product safety model. Do not rely only on application code for tenant isolation, doctor-service consistency, or double-booking prevention.

29.1 Required relational constraints

- Every clinic-owned table must include `clinic_id`.
- Each table referenced by `(clinic_id, id)` must have `UNIQUE (clinic_id, id)`.
- `doctor_id` references must use composite FK: `(clinic_id, doctor_id) -> doctors(clinic_id, id)`.
- `clinic_service_id` references must use composite FK: `(clinic_id, clinic_service_id) -> clinic_services(clinic_id, id)`.
- `doctor_services` must enforce `UNIQUE (clinic_id, doctor_id, clinic_service_id)`.
- `appointment_slots` must include `clinic_id, doctor_id, clinic_service_id, start_time, end_time, status`.
- `appointment_requests` must include `clinic_id, doctor_id, clinic_service_id, slot_id, patient details, reason_for_visit, status, routing_source, is_followup`.

29.2 Slot constraints

Required unique key:

```
UNIQUE (clinic_id, doctor_id, clinic_service_id, start_time, end_time)
```

Recommended overlap protection:

```
EXCLUDE USING gist (  
  clinic_id WITH =,  
  doctor_id WITH =,  
  clinic_service_id WITH =,  
  tstzrange(start_time, end_time, '[)') WITH &&  
)  
WHERE (status IN ('available', 'held', 'pending_confirmation', 'confirmed', 'blocked'));
```

Consistency rules:

- `status='held'` requires `held_by_session_id` and `hold_expires_at`.
- `status='available'` must not have `held_by_session_id`, `hold_expires_at`, or `appointment_request_id`.
- `status in ('pending_confirmation', 'confirmed')` should have `appointment_request_id`.
- held slots expire through background cleanup.

29.3 Appointment status transition table

From status	Allowed to	Who/action	Notes
pending_confirmation	confirmed	admin or owning doctor confirm	Patient notification created only after confirmation.
pending_confirmation	cancelled	admin/owning doctor or approved patient cancel action	Release slot.
pending_confirmation	rescheduled	admin/owning doctor or approved patient reschedule action	Store appointment_event with old/new slot.
confirmed	visited	admin/owning doctor mark visited	visit_reason required; creates patient_visits.
confirmed	cancelled	admin/owning doctor cancel	Release slot; notify patient after action.
confirmed	rescheduled	admin/owning doctor reschedule	Old slot released after new slot held.
confirmed	no_show	admin/owning doctor	No patient_visits row unless configured.
confirmed	conflict_required	schedule/holiday conflict detector	Admin must manually resolve.
conflict_required	confirmed	admin resolves conflict without change	Only if appointment remains valid.
conflict_required	cancelled/rescheduled	admin resolves conflict	Notification event after action.
cancelled	terminal	none	Cannot confirm again. Create new appointment if needed.
visited	terminal	none	Do not cancel visited appointment.
no_show	terminal	none	Create new appointment if patient rebooks.

29.4 Slot status transition table

From	To	Action
available	held	Patient/admin selects slot; SlotService uses row lock.
held	available	Hold expired, user says no, flow cancelled, or confirmation fails.
held	pending_confirmation	Appointment request created in pending mode.
held	confirmed	Appointment created in auto-confirm/manual confirmed mode.
pending_confirmation	confirmed	Clinic confirms appointment.
pending_confirmation/confirmed	available	Appointment cancelled or rescheduled away.
available	blocked	Admin blocks time or generated slot conflicts with schedule/holiday.
blocked	available	Admin unblocks and no overlap exists.

30. Slot Generation and Refresh Policy

Vaidya should use pre-generated appointment slots for the first production version. Dynamic computation on every request is more complex and less safe for locking and dashboard review.

Decision	Final rule
Generation window	Generate slots for the next 45 days for each active doctor-service mapping.
Source	doctor_schedules + doctor_services + clinic_holidays + doctor_blocked_slots.
Slot duration	Default from clinic or service setting; can be overridden per doctor-service later.
Refresh on schedule change	Regenerate only future available/blocked generated slots affected by the schedule change. Never delete pending/confirmed/held slots automatically.
Holiday change	Detect affected appointments. Return conflict list. Do not auto-cancel.
Held slot expiry	Background job releases expired held slots every 1-5 minutes.
Past slots	Do not show for booking. Retain for audit/history if linked to appointments.
Manual override slot	Admin can create appointment outside clinic hours with override reason; no double booking allowed.

31. Auth, Onboarding, and Clinic Membership

Authentication can be Supabase Auth or equivalent JWT provider. Authorization is always enforced by backend RBACGuard using clinic membership and doctor ownership.

31.1 Roles in v1

Role	Purpose	Important limits
platform_admin	Internal Vaidya team operations, support, clinic creation.	Must be highly restricted and audited.
clinic_admin	Clinic owner/reception/admin operations.	Full clinic access except platform-only support tools.
doctor	Doctor self-service.	Own appointments, own timings, own services if allowed. No call inbox. No agent toggle. No knowledge approval.
receptionist	Deferred role.	For v1, clinic_admin represents receptionist/admin operations. Add receptionist later if needed.

31.2 Onboarding flow

25. Platform admin creates clinic record and initial clinic settings.
26. Platform admin invites first clinic_admin user.
27. Clinic admin logs in, completes clinic setup, doctors, services, schedules, fees, and knowledge Q&A.
28. Clinic admin creates doctor profiles. doctor.user_id can be null if doctor does not log in.
29. If doctor login is needed, invite user and link clinic_users.doctor_id to doctors.id.
30. Doctor can switch only among clinics where they have membership, if multi-clinic membership is later enabled.
31. Inactive clinic_users cannot access clinic APIs, even if auth token is valid.

31.3 Patient identity policy

- Phone number alone is not a unique patient identity because family members share phones.
- Primary matching key is clinic_id + normalized_phone + normalized_name when name is known.
- If multiple patients share a phone and name is unknown, Vaidya must ask which patient the appointment is for.
- Do not use previous visit to route to same doctor if patient identity is unclear.
- Normalize phone by stripping spaces, symbols, and consistent country code handling.
- Name matching is case-insensitive but should be conservative for spelling differences. If unsure, ask.

32. Appointment Action Request Policy

This resolves the ambiguity between patient self-service changes and clinic-controlled final actions. By default, Vaidya collects patient cancel/reschedule requests and places them in the admin Reschedule/Cancel section. The admin takes the final operational action. This can later be made configurable per clinic.

Patient request	Default behavior	Why
Cancel appointment by call	Vaidya confirms patient intent, then creates appointment_action_request type=cancel. Appointment is not cancelled until admin approves.	Protects against speech misunderstanding and preserves clinic control.
Reschedule appointment by call	Vaidya collects preferred date/time and creates appointment_action_request type=reschedule or proposes slots depending config. Admin approves final change by default.	Prevents accidental time changes and lets staff manage doctor availability.
Admin/doctor manual cancel	Directly updates appointment after validation and audit.	Authenticated staff action is trusted.
Admin/doctor manual reschedule	Direct transaction with slot lock and notification event after commit.	Staff action with RBAC and audit.

32.1 appointment_action_requests table

```
appointment_action_requests
- id
- clinic_id
- appointment_request_id
- request_type: cancel | reschedule
- requested_by: patient_call | admin | doctor | system
- status: pending | approved | rejected | completed | expired
- requested_new_slot_id nullable
- requested_new_start nullable
- requested_new_end nullable
- reason
- source_session_id nullable
- source_call_id nullable
- created_by_user_id nullable
- reviewed_by_user_id nullable
- reviewed_at nullable
- created_at
- updated_at
```

33. Agent Prompt, JSON Schemas, and Template Registry

The agent is controlled by schemas and templates. Coding agents must not invent random patient-facing text inside handlers. Every common response must be rendered through TemplateRenderer.

33.1 LLM output schema

```
IntentClassification = {
  intent: "book_appointment" | "reschedule_appointment" | "cancel_appointment" |
    "ask_fee" | "ask_timing" | "ask_location" | "ask_doctor_availability" |
    "ask_previsit_instruction" | "ask_insurance" | "ask_human_agent" |
    "emergency" | "medical_advice_request" | "language_switch" | "unknown",
  confidence: number,
  language_style: "ta_tanglish" | "english" | "mixed",
  entities: {
    patient_name?: string | null,
    doctor_name?: string | null,
    reason_for_visit?: string | null,
    date?: string | null,
    time_preference?: "morning" | "afternoon" | "evening" | null,
    visit_type?: "first_visit" | "follow_up" | null,
    topic?: string | null,
    requested_language_style?: "ta_tanglish" | "english" | null
  },
  safety: {
    is_emergency: boolean,
    is_medical_advice_request: boolean,
    reason?: string | null
  },
  needs_clarification: boolean
}
```

33.2 Template key registry - minimum v1

Template key	Tanglish text	English text
booking.greeting	Vanakkam. {clinic_name} Vaidya pesuren. Enna problem-ku appointment venum?	Hello. This is Vaidya from {clinic_name}. What problem do you need the appointment for?
booking.ask_problem_or_doctor	Endha doctor-a paakanum? Illena enna problem-ku appointment venum?	Which doctor would you like to see, or what problem do you need help with?
booking.ask_date	Appointment-ku endha date venum?	Which date would you like the appointment for?
booking.ask_time	Morning, afternoon, illa evening-la edhu venum?	Would you like morning, afternoon, or evening?
booking.propose_slots	{doctor_name} {date_display} {time_display} {slot_list} slots irukku. Edha choose panreenga?	{doctor_name} has {slot_list} slots available {date_display} {time_display}. Which one would you like?
booking.ask_patient_name	Appointment-ku unga peyar sollunga.	Please tell me your name for the appointment.
booking.confirm_details	{patient_name}, {doctor_name} kitta {date_display} {time_display} appointment request create pannalama?	{patient_name}, shall I create the appointment request with {doctor_name} for {date_display} at {time_display}?
booking.created_pending	{patient_name}, unga appointment request create panniten. Clinic staff confirm pannuvanga.	{patient_name}, I have created your appointment request. Clinic staff will confirm it.
booking.flow_cancelled	Okay, appointment request cancel panniten. Vera help venuma?	Okay, I have stopped the appointment request. Do you need anything else?
booking.unsupported_service	Sorry, indha concern-ku clinic staff confirm pannuvanga.	Sorry, clinic staff will confirm this concern.
fee.ask_doctor	Endha doctor-ku fees venum? {doctor_list}	Which doctor fee would you like to know? {doctor_list}
fee.consultation_answer	{doctor_name} consultation fee Rs.{fee}.	{doctor_name}'s consultation fee is Rs.{fee}.
fee.followup_answer	{doctor_name} follow-up fee Rs.{fee}{validity_text}.	{doctor_name}'s follow-up fee is Rs.{fee} {validity_text}.
fee.not_found	Clinic staff confirm pannuvanga.	Clinic staff will confirm this.
knowledge.answer	{answer}	{answer}
knowledge.no_answer	Indha detail clinic staff confirm pannuvanga.	Clinic staff will confirm this.
handoff.ask_reason	Sure. Enna reason-ku clinic staff callback venum?	Sure. What should clinic staff call you about?
handoff.created	Clinic staff unga number-ku callback pannuvanga.	Clinic staff will call you back.
safety.emergency	Idhu emergency-a irukkalaam. Please immediate-a 108-ku call pannunga illa nearest hospital-ku ponga. Clinic staff-ku alert send pannuren.	This may be an emergency. Please call 108 immediately or go to the nearest hospital. I will alert the clinic staff.
safety.medical_advice_refusal	Naan medical advice kudukka mudiyadhu. Doctor consult panna appointment book panna help panren.	I cannot provide medical advice. I can help you book an appointment with the doctor.
unknown.clarify	Sorry, appointment book panna sollreengala, illa clinic details kekkareengala?	Sorry, do you want to book an appointment, or are you asking for clinic details?
language.switched	Peri. Ippo {language_name}-la continue panren.	Sure. I will continue in {language_name}.

34. Structured Versus Knowledge Answer Precedence

Question type	Source of truth	Do not use
Doctor fee / follow-up fee	doctor_fees or fee_rules table	Knowledge base PDF/Q&A if structured fee exists.
Clinic timing / Sunday open	clinic_hours and clinic_holidays	Knowledge base text.
Doctor availability today/tomorrow	appointment_slots first, then doctor_schedules context	Knowledge base text.
Location/address	clinics structured fields	Knowledge base text unless structured field missing.
Service handled/not handled	clinic_services and doctor_services	Knowledge base text.
Parking	clinic_knowledge_base approved Q&A or structured facility field if added	Unapproved answers.
Scan/test preparation	clinic_knowledge_base approved Q&A	LLM guess.
Insurance/cashless explanation	clinic_knowledge_base approved Q&A	LLM guess.
Medical advice	Fixed refusal template	Knowledge base or LLM answer.
Emergency	Fixed emergency template + emergency_incident	Normal booking flow.

35. Frontend Screen-Level Specification

The clinic UI is operational, not technical. It should not expose model confidence, vector score, prompt traces, or routing quality to doctors/receptionists. Those can be internal support tooling later.

35.1 Home tab

Area	Fields/components	Actions
Summary cards	calls_today, pending_confirmations, callback_requests, emergency_alerts, knowledge_gaps, agent_status	Click card filters detailed list.
Needs action list	patient, phone, reason, doctor, requested slot, action type	Confirm, reschedule, call patient, mark done.
Doctor view	own upcoming appointments, own pending confirmations if allowed	Confirm own pending, mark visited own appointments.

35.2 Call inbox tab - admin only

Section	Columns/details	Actions
Call list	call_time, phone, patient_name, duration, outcome, summary, action_needed	Filter by date/outcome/action. Select call.
Detail preview	intent, outcome, linked appointment/callback/emergency, patient info, service, doctor, summary	Open appointment/callback/emergency.
Recording/transcript	audio player if within 10 days, transcript if within 30 days, speaker turns	Play, copy transcript, download only if permission enabled.

35.3 Appointments tab

Section	Columns	Actions and permissions
Pending confirmation	patient_name, phone, doctor, service, reason_for_visit, requested_time, source, followup_badge, status	Admin all. Doctor own only. Confirm, edit time, cancel, call patient, view history.
Confirmed	same as pending plus confirmed_by, confirmation_time	Admin all. Doctor own only. Edit time, cancel, mark visited.
Visited	patient, doctor, service, visit_reason, visited_at, marked_by	Visit reason required. Shows patient history.
Reschedule/cancel	patient action request, current appointment, requested change, source call, status	Admin only approves/rejects patient requested changes.

35.4 Clinic setup tab

Section	Fields	Important rule
Bot answering mode	agent_enabled, answering_mode, fallback_phone, booking_mode, max_concurrent_calls, language, recording_settings	Admin only. Toggle creates audit log. Agent off forwards to fallback number when voice provider supports it.
Doctor timings	doctor, day, start/end, slot_duration, active, blocked slots	Admin all; doctor own only. Conflict check required.
Clinic hours	day, open_time, close_time, active	Admin only. Structured source for timing answers.
Holidays	date, reason, full/partial day	Admin only. Conflict check. No auto-cancel.
Services Vaidya can route	service_name, doctor mapping, fees, handles, does_not_handle, red_flags, examples, active	Structured source for service routing.

35.5 Knowledge base tab

Feature	Behavior
Use template in app	Admin fills Q&A rows directly in the app; no need to download/edit unless preferred.
DOCX upload	Extract Q&A rows into pending_review. Do not import setup fields like fees/schedules/hours.
Manual Q&A	One question + answer + category + optional alternative wording.
Review queue	Bulk approve, single approve, edit, disable.
Approved answer edit	Creates new version and audit log. Regenerate embedding later if vector search enabled.

36. Notification Event Matrix and Retry Policy

Event type	Recipient	Trigger	Send timing
APPOINTMENT_CONFIRMED_PATIENT	patient	Appointment enters confirmed status	After DB commit through async worker.
APPOINTMENT_TIME_CHANGED_PATIENT	patient	Admin/doctor reschedules confirmed appointment	After DB commit.
APPOINTMENT_CANCELLED_PATIENT	patient	Admin/doctor cancels appointment	After DB commit.
PATIENT_CANCEL_REQUEST_ADMIN	clinic admin	Patient requested cancel by call	Immediate admin notification/event.
PATIENT_RESCHEDULE_REQUEST_ADMIN	clinic admin	Patient requested reschedule by call	Immediate admin notification/event.
CALLBACK_REQUEST_ADMIN	clinic admin/reception	Callback request created	Immediate.
EMERGENCY_ALERT_ADMIN	clinic admin/emergency contact	emergency_incident created	Immediate high priority.
KNOWLEDGE_GAP_ADMIN	clinic admin	FAQ unanswered repeatedly	Daily summary or threshold-based.

Notification events must have a deterministic dedupe key such as clinic_id + event_type + appointment_id + status_version. Workers retry with exponential backoff. Suggested attempts: 1 minute, 5 minutes, 30 minutes, 2 hours. After max attempts, mark dead_letter and show in admin/support tools.

37. Voice-Specific Rules for Later Phase

Voice integration must not start until text-core flows pass QA. When voice begins, the following behavior must be implemented before real patient calls.

Voice concern	Default production rule
Greeting/consent	Play clinic greeting and, where required, recording notice before starting conversation.
Silence timeout	If no speech for 5 seconds, ask once again. After two no-speech attempts, offer callback or end politely.
Barge-in	If patient speaks while Vaidya is speaking, stop TTS playback and process patient speech.
ASR low confidence	Ask patient to repeat or simplify. Do not proceed with booking action from low-confidence text.
LLM/provider failure	Use safe fallback: clinic staff will call back. Create callback request if phone exists.
TTS failure	End politely if possible; create callback request/technical alert.
Max call duration	Default 5 minutes for MVP; then transfer/callback.
Agent off	Forward to fallback_phone_number if telephony provider supports.
Recording	Store audio in object storage, not DB. Delete after 10 days.
Latency target	Simple turns under 2.5 seconds perceived latency. Avoid LLM for date/time/yes/no/name/slot parsing.

38. QA Golden Test Matrix

Before production pilot, every item below must pass in QA using text console. Voice tests are added only after text passes.

Category	Test	Expected
Booking	Naalaikku evening appointment venum -> knee pain -> 6:30 -> Kumar -> seri	Creates pending appointment, stores reason, slot linked.
Compound booking	Dr Priya May 16 evening appointment. Name Meena.	Does not re-ask known date/time/name. Checks Dr Priya slots.
Returning patient	Known phone + same problem follow-up	Prefers previous valid doctor and shows history.
Family phone	Known phone with multiple patients and no name	Asks which patient before using history.
Unsupported service	Eye checkup venum for clinic without eye service	No appointment. Staff confirm/unsupported response.
Fee	Fees evlo? then Dr Murugan	Fee clarification flow returns correct doctor fee.
Follow-up fee	Dr Murugan follow-up fee evlo?	Only follow-up fee, not consultation fee too.
Timing	Sunday open-a?	Answers Sunday only, not full schedule.
Doctor availability	Dr Priya inniku irukkangala?	Answers today availability/slots only.
Knowledge approved	Scan-ku fasting venuma?	Uses approved KB answer.
Knowledge pending	Report ready-a? where answer pending_review	Does not use pending answer. Staff confirm.
Tenant isolation	Dental-only FAQ asked in different clinic	No cross-clinic knowledge leakage.
Side question in booking	During booking ask Parking irukka?	Answers FAQ and resumes booking state.
Cancel phrase	During booking say vendam	Flow cancelled, held slot released, no appointment.
Cancel appointment	Patient requests cancel	Creates action request by default or cancels only if configured.
Reschedule	Patient requests time change	Creates/admin action request or validated reschedule as configured.
Handoff	Receptionist kitta pesanum -> reason -> name	Creates pending callback request.
Emergency	Chest pain irukku during any flow	Emergency response, emergency_incident, clinic alert.
Medical advice	Fever-ku enna tablet?	Fixed refusal; no KB or LLM medical answer.
Concurrency	Two sessions choose same slot	Only one hold succeeds. Other gets alternative/no slot.
Idempotency	Repeat same confirmation request	Only one appointment/action created.
RBAC doctor	Doctor tries other doctor appointment	403 DOCTOR_NOT_OWNER.
Retention	Audio >10 days cleanup; transcript >30 days cleanup	Audio deleted; transcript deleted after policy; metadata remains.

39. CI/CD, Migration, and Release Rules

Area	Rule
Branching	main is deployable. Feature branches merge through PR with tests.
Checks	lint, typecheck, unit tests, integration tests, migration dry-run, build must pass.
Migrations	SQL migrations are reviewed. Never let agents auto-push destructive migrations to production.
Rollback	Every release has rollback plan. Schema changes that cannot rollback must be backward-compatible.
QA seeding	QA has fake clinics and deterministic seed data. No real patient data unless approved.
Secrets	Use environment secret manager. Do not commit provider keys.
Production debug	DEBUG API=false. No raw prompts/PII in logs.
Backups	Managed Postgres daily backup. Restore drill before pilot and periodically after.
Object storage	Recording bucket has lifecycle deletion and signed URL access.

40. Security, Privacy, and Access Logs

- All API access requires authentication except explicitly public telephony webhooks; public webhooks must verify provider signatures.
- Every query must be scoped by `clinic_id`.
- Doctor endpoints must enforce owning `doctor_id`.
- Record access to call recordings and transcripts: `user_id`, `clinic_id`, `call_id`, timestamp, purpose if provided.
- Use short-lived signed URLs for recordings. Never expose raw storage URLs.
- Mask phone numbers in application logs unless debug is explicitly needed in QA.
- Do not log full transcripts to normal application logs. Store transcripts in DB with retention policy.
- Support/platform admin access must be audited and should require reason for viewing clinic data.
- Add privacy policy and recording notice before real voice pilot.
- Normal UI must not hard-delete patient/appointment/knowledge/call records. Use disabled/cancelled/archived/expired statuses.

41. Plan, Usage, and Concurrency Controls

Plan field	Purpose
included_minutes	Monthly included voice minutes per clinic.
max_concurrent_calls	Controls simultaneous voice sessions per clinic.
recording_retention_days	Default 10; do not exceed plan/policy.
transcript_retention_days	Default 30.
booking_mode	pending_confirmation or auto_confirm.
auto_confirm_allowed	Whether clinic can enable auto-confirm.
overage_rate	Future billing for minutes beyond plan.
agent_enabled	Admin-controlled runtime switch.

Initial practical concurrency limits: pilot clinics 1-2 concurrent calls, Pro 3 concurrent calls, Clinic+ 5 concurrent calls, custom after load testing. Do not promise unlimited or 50 simultaneous calls until telephony, STT, LLM, TTS, backend, DB, and notification load have been tested.

42. Final Production Readiness Checklist

Area	Ready condition
Architecture	Modular NestJS backend, provider adapters, no direct DB access from LLM.
RBAC	Admin/doctor scopes enforced by backend. Doctor own data only.
Agent safety	LLM returns JSON only; backend validates and acts.
Booking	Problem/reason stored; slot locked; pending confirmation default.
Follow-up	Patient history used only after identity and same-problem validation.
Cancel/reschedule	Admin action request by default for patient call changes; staff action audited.
DB integrity	Composite FKs, row locks, indexes, constraints, transition rules.
Knowledge	Approved Q&A only; structured facts from structured tables.
Notifications	Outbox/retry; patient confirm only after confirmed appointment.
Recording/transcript	Audio 10 days, transcript 30 days, signed URLs, access logs.
UI	Five-tab workflow; admin-only call inbox; no AI debug screens for clinics.
QA	Golden tests pass; concurrency/idempotency tests pass.
Deployment	QA/prod separated; backups; migration review; rollback plan.
Voice	Only after text QA; silence/barge-in/failure/latency rules implemented.

Final implementation instruction: if any coding agent finds a conflict between the high-level implementation plan and this document, this final document wins. If a new business decision changes behavior, update this document first and then update schema/API/tests accordingly.